Strategy Design Pattern — Interchangeable Algorithms at Runtime

Problem Statement (motivating scenario)

Primary scenario (List printing and sorting): You are asked to build a simple integer list type with the following API:

`reverse()`, `get(i)`, `set(i, a)`, `sort()`, `print()`.

Two early clients disagree on printing:

- *Horizontal print*: all elements on one line, space-separated.
- *Vertical print*: one element per line.

A clean solution is:

1. Create an abstract base `MyIntList` with concrete `reverse/get/set/sort` and an *abstract* `print()`.
2. Implement `sort()` with a general-purpose algorithm (e.g., mergesort/quicksort).
3. Provide two subclasses: `HorizontalPrintList` and `VerticalPrintList`, each overriding `print()`.

This is *good enough* for the initial printing variability.

However, four more clients arrive, each with **data-specific sorting** constraints:

- **Client 1**: list will contain *only* 0 and 1 ⇒ optimal *counting/single-pass* partition instead of mergesort/quicksort.
- **Client 2**: list will be *almost sorted* ⇒ *insertion sort* is usually best.
- **Client 3**: list is *already sorted descending* ⇒ *just reverse* for ascending output.
- **Client 4**: *completely random* ⇒ *quicksort/mergesort* is fine.

Question: Do we create more subclasses or add another inheritance layer per sorting variant? We'll examine why that *doesn't* scale and build toward Strategy.

Secondary scenario 1: Payment/checkout fee computation. A platform must compute checkout fees using different policies (`FixedFee`, `PercentageFee`, `TieredFee`, `MixedFee`), selectable at runtime per country/plan and swappable for A/B tests.

Secondary scenario 2: Game AI steering/pathfinding. NPCs select movement algorithms (`Seek`, `Evade`) or pathfinding variants (`AStarGrid`, `NavMeshAStar`) at runtime per NPC or difficulty.

Forces, Constraints, Goals

Across all scenarios:

- **Runtime selection**: tenant policies, level design, and experiments must toggle behaviors without code edits.
- **Testability and rollout**: a crisp interface lets us unit-test each algorithm and safely flip it via config/feature flags.

- **Performance/determinism**: choose the fastest correct algorithm for the *data shape* (binary data, nearly sorted, etc.); preserve deterministic behavior (especially in games) and stable rounding (in money).
- **Configuration**: inject immutable parameters; avoid mutable global state.

Naïve Alternatives and Why They Fail (with SOLID references)

Explosion of subclasses / deeper inheritance hierarchies

Suppose we start with:

$$\texttt{MyIntList} \rightarrow \{\texttt{HorizontalPrintList}, \texttt{VerticalPrintList}\}$$

To add sort specialization per client, we might try subclasses like `HorizontalBinaryList`, `VerticalBinaryList`, `HorizontalAlmostSortedList`, ... This leads to a **combinatorial explosion** as we cross-printing modes (2) with sort variants (4+), then with future axes (e.g., logging, bounds checks). It violates SRP (each subclass mixes printing *and* sorting policy) and harms OCP (new algorithm forces many new classes).

Listing 1: Anti-pattern: behavior gridlocked in inheritance

```
abstract class MyIntListBase { /* reverse/get/set/sort/print... */ }
class HorizontalPrintList extends MyIntListBase { /* print horizontal */ }
class VerticalPrintList   extends MyIntListBase { /* print vertical   */ }

// New requirements:
class HorizontalBinaryList extends HorizontalPrintList { /* override sort()
    w/ count */ }
class VerticalBinaryList   extends VerticalPrintList   { /* override sort()
    w/ count */ }
// ... and so on for each combination (almost-sorted, desc, random)
```

Giant `if/else` inside `sort()`

Placing all data-shape branches inside one `sort()` violates SRP and OCP; it's hard to test and invites *shotgun surgery*.

Listing 2: Anti-pattern: one method knows every algorithm

```
class MyIntListBad {
  void sort(String dataProfile) {
    if ("BINARY".equals(dataProfile)) { /* count sort */ }
    else if ("ALMOST".equals(dataProfile)) { /* insertion */ }
    else if ("DESC".equals(dataProfile)) { /* reverse */ }
    else { /* quick/merge */ }
  }
}
```

Static "god" utils

A `SortUtils.sortByProfile(list, profile)` centralizes logic but couples clients to a concrete utility (hurts DIP), hides state, and is awkward to mock.

Strategy: Core Idea and Intuition

Definition. Encapsulate a family of algorithms, make them *interchangeable*, and let the host object (the *context*) hold a reference to one *strategy* and delegate work to it. Clients depend on *the interface*, not concrete algorithms.

Intuition for the list problem. Keep printing as a *class specialization* (two small subclasses for horizontal/vertical), but *extract sorting* behind `SortStrategy`. Now you can choose the optimal sorter (counting, insertion, reverse, quick/merge) at runtime per client or even per call.

Design Heuristics

- Keep the strategy interface *minimal*: e.g., `void sort(List<Integer> data)` for the list, `Money fee(...)` for checkout, `Vector2 steer(...)` for AI.
- Prefer stateless strategies with immutable configuration.
- Select via constructor injection, factory/registry, or configuration mapping; avoid `instanceof` in clients.
- Document pre/postconditions (e.g., stable rounding; in-place sort).
- Add telemetry hooks (measure latency per strategy).
- Be explicit about time/space complexity and data-shape assumptions.

Concrete Java for the List Scenario

We first show the *printing variability* with inheritance (which is fine), then refactor *sorting* into strategies.

Phase A — Printing via small subclasses

Listing 3: MyIntList.java – abstract base with default mergesort

```java
import java.util.*;

public abstract class MyIntList {
  protected final List<Integer> data = new ArrayList<>();

  public void add(int x) { data.add(x); }
  public int  get(int i) { return data.get(i); }
  public void set(int i, int a) { data.set(i, a); }

  public void reverse() {
    Collections.reverse(data);
  }

  /** Default general-purpose sort (e.g., mergesort via Collections.sort
      for stability). */
  public void sort() {
    Collections.sort(data); // TimSort (merge-like) in OpenJDK; stable and
        fast on runs.
  }
```

```
  /** Abstract printing policy. */
  public abstract void print();
}
```

Listing 4: HorizontalPrintList.java

```
public class HorizontalPrintList extends MyIntList {
  @Override public void print() {
    for (int i = 0; i < data.size(); i++) {
      System.out.print(data.get(i));
      if (i + 1 < data.size()) System.out.print(" ");
    }
    System.out.println();
  }
}
```

Listing 5: VerticalPrintList.java

```
public class VerticalPrintList extends MyIntList {
  @Override public void print() {
    for (int x : data) System.out.println(x);
  }
}
```

Listing 6: DemoPrinting.java

```
public class DemoPrinting {
  public static void main(String[] args) {
    MyIntList h = new HorizontalPrintList();
    MyIntList v = new VerticalPrintList();
    for (int x : new int[]{3,1,4,1,5}) { h.add(x); v.add(x); }

    h.sort(); h.print();
    v.sort(); v.print();
  }
}
```

This solves printing nicely. But per-client sorting optimality still pushes us toward Strategy.

Phase B — Sorting via `SortStrategy`

Listing 7: SortStrategy.java

```
import java.util.List;

public interface SortStrategy {
  /** Sort the list in-place ascending. Implementations must be
     deterministic. */
  void sort(List<Integer> data);
  default String name() { return getClass().getSimpleName(); }
}
```

Listing 8: CountSort01.java – for data in {0,1}

```
import java.util.List;

public final class CountSort01 implements SortStrategy {
  @Override public void sort(List<Integer> data) {
```

```
    int zeros = 0;
    for (int x : data) if (x == 0) zeros++;
    for (int i = 0; i < data.size(); i++) data.set(i, i < zeros ? 0 : 1);
  }
}
```

Listing 9: InsertionSort.java – great for nearly-sorted data

```
import java.util.List;

public final class InsertionSort implements SortStrategy {
  @Override public void sort(List<Integer> data) {
    for (int i = 1; i < data.size(); i++) {
      int key = data.get(i);
      int j = i - 1;
      while (j >= 0 && data.get(j) > key) {
        data.set(j + 1, data.get(j));
        j--;
      }
      data.set(j + 1, key);
    }
  }
}
```

Listing 10: ReverseAlreadyDesc.java – if list is known to be descending

```
import java.util.Collections;
import java.util.List;

public final class ReverseAlreadyDesc implements SortStrategy {
  @Override public void sort(List<Integer> data) {
    // Preconditions: data already sorted descending
    Collections.reverse(data);
  }
}
```

Listing 11: QuickSort.java – general-purpose for random data

```
import java.util.List;
import java.util.Random;

public final class QuickSort implements SortStrategy {
  private final Random rnd = new Random(42); // deterministic pivoting

  @Override public void sort(List<Integer> a) {
    qsort(a, 0, a.size() - 1);
  }
  private void qsort(List<Integer> a, int lo, int hi) {
    if (lo >= hi) return;
    int p = lo + rnd.nextInt(hi - lo + 1);
    int pivot = a.get(p);
    int i = lo, j = hi;
    while (i <= j) {
      while (a.get(i) < pivot) i++;
      while (a.get(j) > pivot) j--;
      if (i <= j) { int tmp = a.get(i); a.set(i, a.get(j)); a.set(j, tmp);
          i++; j--; }
    }
```

```
    if (lo < j) qsort(a, lo, j);
    if (i < hi) qsort(a, i, hi);
  }
}
```

Listing 12: StrategyEnabledList.java – composition over inheritance

```
import java.util.Objects;

public class StrategyEnabledList extends HorizontalPrintList {
  private SortStrategy sorter;

  public StrategyEnabledList(SortStrategy initial) {
    this.sorter = Objects.requireNonNull(initial);
  }

  public void setSorter(SortStrategy s) { this.sorter = Objects.
      requireNonNull(s); }

  @Override public void sort() {
    long t0 = System.nanoTime();
    sorter.sort(this.data);
    long t1 = System.nanoTime();
    System.out.printf("[Sort] %-20s took %.3f ms%n", sorter.name(), (t1 -
        t0)/1e6);
  }
}
```

Listing 13: DemoSortingStrategies.java

```
public class DemoSortingStrategies {
  public static void main(String[] args) {
    // Client 1: binary data
    StrategyEnabledList bin = new StrategyEnabledList(new CountSort01());
    for (int x : new int[]{1,0,1,1,0,0,1}) bin.add(x);
    bin.sort(); bin.print(); // fast counting pass

    // Client 2: almost sorted
    StrategyEnabledList near = new StrategyEnabledList(new InsertionSort())
        ;
    for (int x : new int[]{1,2,3,5,4,6,7}) near.add(x);
    near.sort(); near.print();

    // Client 3: known descending
    StrategyEnabledList desc = new StrategyEnabledList(new
        ReverseAlreadyDesc());
    for (int x : new int[]{9,8,7,6,5}) desc.add(x);
    desc.sort(); desc.print();

    // Client 4: random
    StrategyEnabledList rnd = new StrategyEnabledList(new QuickSort());
    for (int x : new int[]{9,3,7,1,8,2,5}) rnd.add(x);
    rnd.sort(); rnd.print();

    // \emph{Hot-swap} at runtime:
    rnd.setSorter(new InsertionSort()); // if we know it's almost-sorted
        next time
  }
```

```
}
```

Why this is better than deeper inheritance. Printing remains a small, stable specialization (two subclasses). Sorting is a separate axis of variability, so we *compose* it via `SortStrategy`. We avoid subclass multiplication, adhere to SRP (each strategy owns one algorithm), and support runtime switching (experiments, client-specific tuning).

Detailed Java Example (checkout fee strategies)

We now keep the earlier checkout example unchanged to reinforce Strategy in a different domain.

Money value object and checkout context

Listing 14: Money.java

```java
// Money.java
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.Objects;

/** Immutable money (currency + amount) with canonical 2-decimal rounding.
   */
public final class Money implements Comparable<Money> {
  private final String currency;
  private final BigDecimal amount;

  public Money(String currency, BigDecimal amount) {
    this.currency = Objects.requireNonNull(currency);
    this.amount = amount.setScale(2, RoundingMode.HALF_UP);
  }

  public static Money of(String currency, String amount) {
    return new Money(currency, new BigDecimal(amount));
  }

  public static Money zero(String currency) { return new Money(currency,
      BigDecimal.ZERO); }

  public String currency() { return currency; }
  public BigDecimal amount() { return amount; }

  public Money plus(Money other) {
    requireSameCurrency(other);
    return new Money(currency, amount.add(other.amount));
  }

  public Money minus(Money other) {
    requireSameCurrency(other);
    return new Money(currency, amount.subtract(other.amount));
  }

  /** Multiply by a percent: 2.9 -> 2.9% */
  public Money percent(BigDecimal pct) {
    BigDecimal factor = pct.movePointLeft(2); // 2.9 -> 0.029
    return new Money(currency, amount.multiply(factor));
```

```java
  }

  public Money multiply(BigDecimal factor) {
    return new Money(currency, amount.multiply(factor));
  }

  public Money max(Money other) {
    requireSameCurrency(other);
    return this.compareTo(other) >= 0 ? this : other;
  }

  @Override public int compareTo(Money o) {
    requireSameCurrency(o);
    return this.amount.compareTo(o.amount);
  }

  private void requireSameCurrency(Money other) {
    if (!this.currency.equals(other.currency))
      throw new IllegalArgumentException("Currency mismatch: " + this.
          currency + " vs " + other.currency);
  }

  @Override public String toString() { return currency + " " + amount; }
}
```

Listing 15: CheckoutContext.java

```java
// CheckoutContext.java
import java.util.Objects;

public final class CheckoutContext {
  private final String country;      // ISO-3166-1 alpha-2, e.g., "US"
  private final String currency;     // ISO-4217, e.g., "USD"
  private final String gateway;      // e.g., "Stripe", "Adyen"
  private final String merchantPlan; // e.g., "BASIC", "PRO", "ENTERPRISE"
  private final Money  cartSubtotal;

  public CheckoutContext(String country, String currency, String gateway,
                         String merchantPlan, Money cartSubtotal) {
    this.country = Objects.requireNonNull(country);
    this.currency = Objects.requireNonNull(currency);
    this.gateway = Objects.requireNonNull(gateway);
    this.merchantPlan = Objects.requireNonNull(merchantPlan);
    this.cartSubtotal = Objects.requireNonNull(cartSubtotal);
  }

  public String country()      { return country; }
  public String currency()     { return currency; }
  public String gateway()      { return gateway; }
  public String merchantPlan() { return merchantPlan; }
  public Money  subtotal()     { return cartSubtotal; }

  @Override public String toString() {
    return "ctx{" + country + "," + currency + "," + gateway + "," +
        merchantPlan + ", subtotal=" + cartSubtotal + "}";
  }
}
```

Strategy interface and concrete strategies

Listing 16: FeeStrategy.java

```java
// FeeStrategy.java
public interface FeeStrategy {
  /** Compute the fee charged to the merchant. Must be >= 0 and same
     currency as subtotal. */
  Money fee(Money subtotal, CheckoutContext ctx);
  /** Optional: a short label for telemetry. */
  default String name() { return getClass().getSimpleName(); }
}
```

Listing 17: FixedFee.java

```java
// FixedFee.java
import java.math.BigDecimal;

public final class FixedFee implements FeeStrategy {
  private final Money fixed;

  public FixedFee(Money fixed) {
    if (fixed.amount().compareTo(BigDecimal.ZERO) < 0)
      throw new IllegalArgumentException("fixed fee must be >= 0");
    this.fixed = fixed;
  }

  @Override public Money fee(Money subtotal, CheckoutContext ctx) {
    return Money.of(subtotal.currency(), fixed.amount().toPlainString());
  }
}
```

Listing 18: PercentageFee.java

```java
// PercentageFee.java
import java.math.BigDecimal;

public final class PercentageFee implements FeeStrategy {
  private final BigDecimal pct; // e.g., 2.9 means 2.9%

  public PercentageFee(BigDecimal pct) {
    if (pct.compareTo(BigDecimal.ZERO) < 0) throw new
       IllegalArgumentException("pct >= 0");
    this.pct = pct;
  }

  @Override public Money fee(Money subtotal, CheckoutContext ctx) {
    return subtotal.percent(pct);
  }
}
```

Listing 19: TieredFee.java

```java
// TieredFee.java
import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;
import java.util.Objects;
```

```java
/** Piecewise percentage schedule: for amount in [threshold, next), apply
    its pct. */
public final class TieredFee implements FeeStrategy {
  public static final class Tier {
    public final BigDecimal upToInclusive; // e.g., 100.00
    public final BigDecimal pct;           // e.g., 1.9 for 1.9%
    public Tier(BigDecimal upToInclusive, BigDecimal pct) {
      this.upToInclusive = upToInclusive; this.pct = pct;
    }
  }

  private final List<Tier> tiers = new ArrayList<>();
  private final BigDecimal lastPct;

  public TieredFee(List<Tier> tiersOrderedByLimit, BigDecimal overflowPct)
      {
    Objects.requireNonNull(tiersOrderedByLimit);
    if (tiersOrderedByLimit.isEmpty()) throw new IllegalArgumentException("
        tiers required");
    this.tiers.addAll(tiersOrderedByLimit);
    this.lastPct = Objects.requireNonNull(overflowPct);
  }

  @Override public Money fee(Money subtotal, CheckoutContext ctx) {
    BigDecimal remaining = subtotal.amount();
    BigDecimal acc = BigDecimal.ZERO;
    BigDecimal prev = BigDecimal.ZERO;

    for (Tier t : tiers) {
      BigDecimal span = t.upToInclusive.subtract(prev);
      BigDecimal take = remaining.min(span).max(BigDecimal.ZERO);
      if (take.signum() > 0) {
        acc = acc.add(take.multiply(t.pct.movePointLeft(2)));
        remaining = remaining.subtract(take);
      }
      prev = t.upToInclusive;
      if (remaining.signum() <= 0) break;
    }
    // overflow
    if (remaining.signum() > 0) {
      acc = acc.add(remaining.multiply(lastPct.movePointLeft(2)));
    }
    return new Money(subtotal.currency(), acc);
  }
}
```

Listing 20: MixedFee.java

```java
// MixedFee.java
/** Example composition: maximum of a percentage fee and a fixed floor. */
public final class MixedFee implements FeeStrategy {
  private final PercentageFee percentage;
  private final FixedFee floor;

  public MixedFee(PercentageFee percentage, FixedFee floor) {
    this.percentage = percentage; this.floor = floor;
  }
```

```
  @Override public Money fee(Money subtotal, CheckoutContext ctx) {
    Money a = percentage.fee(subtotal, ctx);
    Money b = floor.fee(subtotal, ctx);
    return a.max(b);
  }
}
```

Context, registry, and demo

Listing 21: StrategyRegistry.java

```
// StrategyRegistry.java
import java.util.Map;
import java.util.Objects;
import java.util.concurrent.ConcurrentHashMap;

/** Simple runtime registry mapping (country, plan) -> FeeStrategy. */
public final class StrategyRegistry {
  private final Map<String, FeeStrategy> byKey = new ConcurrentHashMap<>();

  private static String key(String country, String plan) {
    return country + "::" + plan;
  }

  public void register(String country, String plan, FeeStrategy s) {
    byKey.put(key(country, plan), Objects.requireNonNull(s));
  }

  public FeeStrategy resolve(CheckoutContext ctx, FeeStrategy fallback) {
    return byKey.getOrDefault(key(ctx.country(), ctx.merchantPlan()),
        fallback);
  }
}
```

Listing 22: FeeEngine.java

```
// FeeEngine.java
import java.util.Objects;

public final class FeeEngine {
  private FeeStrategy strategy;        // current policy
  private final StrategyRegistry reg;  // selection source (optional)

  public FeeEngine(FeeStrategy initial, StrategyRegistry reg) {
    this.strategy = Objects.requireNonNull(initial);
    this.reg = reg;
  }

  public void setStrategy(FeeStrategy s) { this.strategy = Objects.
      requireNonNull(s); }

  public Money compute(CheckoutContext ctx) {
    FeeStrategy s = (reg != null) ? reg.resolve(ctx, strategy) : strategy;
    long t0 = System.nanoTime();
    Money fee = s.fee(ctx.subtotal(), ctx);
    long t1 = System.nanoTime();
```

```
    System.out.printf("[FeeEngine] %-16s fee=%s latency=%.3fms%n", s.name()
        , fee, (t1-t0)/1e6);
    return fee;
  }
}
```

Listing 23: MainDemo.java

```
// MainDemo.java
import java.math.BigDecimal;
import java.util.List;

public class MainDemo {
  public static void main(String[] args) {
    Money subtotal = Money.of("USD", "125.00");
    CheckoutContext ctxUSBasic = new CheckoutContext("US", "USD", "Stripe",
        "BASIC", subtotal);
    CheckoutContext ctxUSPro   = new CheckoutContext("US", "USD", "Stripe",
        "PRO",   subtotal);

    // Define strategies
    FeeStrategy fixed30c = new FixedFee(Money.of("USD", "0.30"));
    FeeStrategy pct29    = new PercentageFee(new BigDecimal("2.9"));
    FeeStrategy mixed    = new MixedFee(new PercentageFee(new BigDecimal("
       1.9")), new FixedFee(Money.of("USD","0.50")));
    FeeStrategy tiered   = new TieredFee(
      List.of(
        new TieredFee.Tier(new BigDecimal("100.00"), new BigDecimal("1.9"))
            ,
        new TieredFee.Tier(new BigDecimal("200.00"), new BigDecimal("1.7"))
      ),
      new BigDecimal("1.5")
    );

    // Registry mapping country+plan to strategies
    StrategyRegistry reg = new StrategyRegistry();
    reg.register("US", "BASIC", pct29);
    reg.register("US", "PRO",   mixed);

    // Engine with default strategy and registry
    FeeEngine engine = new FeeEngine(fixed30c, reg);

    // Compute under registry-driven selection
    engine.compute(ctxUSBasic); // PercentageFee(2.9%)
    engine.compute(ctxUSPro);   // MixedFee(max(1.9%, 0.50))

    // Hot-swap at runtime
    System.out.println("-- swap to tiered for everyone --");
    engine.setStrategy(tiered);
    CheckoutContext ctxCAEnt = new CheckoutContext("CA", "USD", "Adyen", "
       ENTERPRISE", subtotal);
    engine.compute(ctxCAEnt);  // uses TieredFee as fallback
  }
}
```

Secondary Java Example (game development)

Listing 24: Vector2.java

```java
// Vector2.java
public final class Vector2 {
  public final double x, y;
  public Vector2(double x, double y) { this.x = x; this.y = y; }
  public Vector2 add(Vector2 o) { return new Vector2(x + o.x, y + o.y); }
  public Vector2 sub(Vector2 o) { return new Vector2(x - o.x, y - o.y); }
  public Vector2 scale(double s) { return new Vector2(x * s, y * s); }
  public double length() { return Math.hypot(x, y); }
  public Vector2 normalized() { double L = length(); return L == 0 ? this :
      scale(1.0/L); }
  @Override public String toString() { return String.format("(%.2f, %.2f)",
      x, y); }
}
```

Listing 25: SteeringState.java

```java
// SteeringState.java
public final class SteeringState {
  public final Vector2 position;
  public final Vector2 target;
  public final Vector2 threat;
  public final double  maxAccel;
  public SteeringState(Vector2 position, Vector2 target, Vector2 threat,
     double maxAccel) {
    this.position = position; this.target = target; this.threat = threat;
       this.maxAccel = maxAccel;
  }
}
```

Listing 26: SteeringStrategy.java

```java
// SteeringStrategy.java
public interface SteeringStrategy {
  /** Return desired acceleration vector. Deterministic per state. */
  Vector2 steer(SteeringState s);
  default String name() { return getClass().getSimpleName(); }
}
```

Listing 27: Seek.java

```java
// Seek.java
public final class Seek implements SteeringStrategy {
  @Override public Vector2 steer(SteeringState s) {
    Vector2 dir = s.target.sub(s.position).normalized();
    return dir.scale(s.maxAccel);
  }
}
```

Listing 28: Evade.java

```java
// Evade.java
public final class Evade implements SteeringStrategy {
  @Override public Vector2 steer(SteeringState s) {
    Vector2 away = s.position.sub(s.threat).normalized();
    return away.scale(s.maxAccel);
  }
}
```

Listing 29: Mover.java

```java
// Mover.java
public final class Mover {
  private Vector2 velocity = new Vector2(0, 0);
  private SteeringStrategy strategy;

  public Mover(SteeringStrategy initial) { this.strategy = initial; }
  public void setStrategy(SteeringStrategy s) { this.strategy = s; }

  public Vector2 update(SteeringState state, double dt) {
    Vector2 accel = strategy.steer(state);
    velocity = velocity.add(accel.scale(dt));
    return state.position.add(velocity.scale(dt));
  }
}
```

Listing 30: GameDemo.java

```java
// GameDemo.java
public class GameDemo {
  public static void main(String[] args) {
    SteeringStrategy seek  = new Seek();
    SteeringStrategy evade = new Evade();
    Mover npc = new Mover(seek);

    SteeringState state = new SteeringState(
      new Vector2(0,0),
      new Vector2(10,0), // target
      new Vector2(-5,0), // threat
      5.0                // max acceleration
    );

    // Move with Seek
    Vector2 pos1 = npc.update(state, 0.1);
    System.out.println(seek.name() + " pos=" + pos1);

    // Low health? Swap to Evade at runtime
    npc.setStrategy(evade);
    Vector2 pos2 = npc.update(new SteeringState(pos1, state.target, state.
        threat, 5.0), 0.1);
    System.out.println(evade.name() + " pos=" + pos2);
  }
}
```

Valid doubt: Is Strategy the same as State? / Answer:

Valid doubt: "Isn't this the same as *State*?"
Answer: *No.* Strategy externalizes *which* algorithm to apply (chosen by configuration or context). *State* models an object's internal state machine; behavior changes *as it transitions.* You may change a Strategy arbitrarily; State transitions are constrained by the state model.

Valid doubt: How is Strategy different from Template Method? / Answer:

Valid doubt: "How is this different from *Template Method*?"
Answer: Template Method fixes the skeleton of an algorithm in an abstract base class with overridable steps (inheritance). Strategy replaces the entire algorithm behind a stable interface (composition), enabling runtime swapping and reducing inheritance coupling.

Valid doubt: Why not Decorator? / Answer:

Valid doubt: "Why not *Decorator*?"
Answer: Decorator *stacks* behaviors (often many at once). Strategy selects *one* algorithm. In our list scenario, using Decorator to add "counting" on top of "quicksort" would double-sort; we need a single chosen policy.

Valid doubt: Different exceptions/return shapes? / Answer:

Valid doubt: "Can strategies throw different exceptions or return different shapes?"
Answer: Maintain a consistent contract (LSP). For sorting, all strategies must sort ascending in-place and be deterministic; for fees, always return non-negative `Money` in the same currency. Expose diagnostics via telemetry instead of changing signatures.

Testing Strategy

List strategies: property tests (idempotence of sort; monotonicity; stability expectations where relevant), edge cases (empty, singleton, all equal, already sorted, reverse-sorted).
Fees: contract tests (non-negative, currency preservation), golden cases for tiers, and micro-benchmarks per strategy.

Listing 31: TieredFeeTest.java (JUnit 5 snippet)

```java
// TieredFeeTest.java
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;
import java.math.BigDecimal;
import java.util.List;

class TieredFeeTest {
  @Test void thresholds_are_applied_correctly() {
    TieredFee tf = new TieredFee(
      List.of(
        new TieredFee.Tier(new BigDecimal("100.00"), new BigDecimal("1.0"))
            , // 1% up to 100
        new TieredFee.Tier(new BigDecimal("200.00"), new BigDecimal("2.0"))
              // 2% up to 200
      ),
      new BigDecimal("3.0") // 3% beyond
    );
    Money fee150 = tf.fee(Money.of("USD","150.00"),
        new CheckoutContext("US","USD","G","BASIC", Money.of("USD","150.00"
            )));
    // 100*1% + 50*2% = 1.00 + 1.00 = 2.00
    assertEquals("USD 2.00", fee150.toString());
  }
```

```
}
```

Mapping to SOLID

SRP. Each strategy class owns one algorithm (printing subclasses only print; `SortStrategy` only sorts).
OCP. Add a new sorting or fee policy without modifying the clients or contexts.
LSP. All strategies respect the same contract (sorted ascending; non-negative fees).
ISP. Lean interfaces (`sort(List<Integer>)`, `fee(...)`).
DIP. Clients depend on `SortStrategy`/`FeeStrategy`/`SteeringStrategy`, not concretes.

Pitfalls and How to Avoid Them

Leaky selection logic. Centralize mapping from context to strategy in a registry/factory.
Hidden mutability. Keep configuration immutable; seed any randomness for determinism.
Units/rounding bugs. Define canonical rounding policy (see `Money`).
Strategy proliferation. Assign ownership, regularly prune/merge similar strategies.
Overuse. If you never change the algorithm, Strategy might be over-engineering (YAGNI).

When to Use / When Not to Use

Use when behavior must be selected at runtime, per tenant/level/data shape, or when running A/B tests.
Avoid when you need composed cross-cutting behavior (Decorator), workflow coordination (Facade), interface translation (Adapter), or when there's only ever one algorithm.

Mini Walkthrough / Exercise

List:

1. Start with `MyIntList` + `HorizontalPrintList`/`VerticalPrintList`; verify printing variants.
2. Introduce `SortStrategy` and wire `StrategyEnabledList`; run the four client profiles using `CountSort01`, `InsertionSort`, `ReverseAlreadyDesc`, `QuickSort`.
3. Hot-swap sorters at runtime and record timings.

Checkout: run `MainDemo` and flip registry mappings.
Game: start NPC with `Seek`; switch to `Evade` when health drops; observe trajectory change.

Takeaway

Strategy separates *policy* from *mechanics*. In the list example, printing is a simple subclass choice, while sorting becomes a plug-in policy chosen for data shape and switched at runtime. In payments and games, the same idea prevents `if/else` bloat, enables safe experiments, and keeps clients stable as algorithms evolve. Small interface, big leverage.